

# SCHEDULE TRACKER

Eula Ryz B. Baroga  
8-Rosal

## **II. PROBLEM STATEMENT**

People across various routines often struggle to organize their daily schedules that manage their priorities and balance overlapping commitments. This leads to disorganization, missed deadlines, and stress. This project aims to create an organized schedule tracker that helps people with their routine.

## **III. PROJECT OBJECTIVES**

### **SPECIFIC & MEASURABLE**

- Develop a flexible schedule tracker that enables users to create, organize, and complete daily tasks and time based reminders.

### **ATTAINABLE & REALISTIC**

- Implement an automated priority sorting algorithm to organize tasks by due date and urgency level.

### **TIME BOUND**

- Finalize the project proposal, computational design, and interface layout within four weeks.

## **IV. PLANNED FEATURES**

### **UNIVERSAL TASK & SCHEDULE MANAGEMENT**

- Add, update, and remove tasks with customizable categories and target due dates.

### **CHRONOLOGICAL SCHEDULE SORTING**

- Automatically display events in strict chronological order from morning to evening.

### **CONFLICT DETECTION**

- Flag overlapping time blocks to prevent double-booking.

## **V. PLANNED INPUTS & OUTPUTS**

### **INPUTS**

- Event/Routine title, start time, end time, and frequency
- User adjustments (Rescheduling time slots, canceling scheduled events)

### **OUTPUTS**

- A visual, time-blocked daily agenda or timetable view.
- Visual time indicators (Current time marker, color-coded event categories, highlight warnings for overlapping schedules).

## **VI. LOGIC PLAN**

Start the main program loop so the user can perform multiple actions until they choose to leave.

Display the main menu with three options: Add Schedule Entry, View Daily Timetable, or Exit.

Ask the user to enter their choice (1, 2, or 3).

If the user chooses Option 1 (Add Entry):

Ask the user to enter the event's title, start time, end time, and category.

Check every existing event in the schedule list to see if the new event's time overlaps with it.

If an overlap is found, display a conflict warning and ask if they still want to save it.

If the user agrees (or if there is no conflict), create the new event and save it to the schedule list.

Return to the main menu.

If the user chooses Option 2 (View Timetable):

Check if the schedule list is empty. If so, display "No events scheduled for today."

If there are events, sort the list chronologically by start time.

Loop through the list and print each event's details (start time, end time, title, and category).

Return to the main menu.

If the user chooses Option 3 (Exit):

Display a farewell message and stop the program.

If the user enters any other choice:

Display an invalid choice error message and return to the main menu prompt.